EMERGING TECHNOLOGIES | TEAM SPEAKER NOTES

Presentation speaking order

Mai Tuấn Dũng

English narration for assigned slides, in the classroom display order.

PRESENTER	ROLE	SLIDES	COVERAGE
Trần Việt Hải	Opening	**1–4**	Introduction, problem, project goals
Lâm Xê Khoa	Main presenter 1	**5–15**	LLMs, RAG, architecture, source PDFs, metadata
Mai Tuấn Dũng	Main presenter 2	**16–26**	Chunking, embeddings, FAISS, pipelines, LCEL, prompting
Sơn Nguyên	Main presenter 3	**27–37**	Ollama, app.py, logging, evaluation, risks, conclusion
Châu Ngọc Đức	Demo lead	**38**	Observed LangChain question; launch/operate demo
Trần Việt Hải	Demo + closing	**39–40**	Out-of-PDF question; instructor questions; closing

Display order

Slides 1–30 → original slides 33–39 → original slides 31–32 → final Questions slide 40. The presentation-order PowerPoint already reflects this sequence.

Duc operates the Windows 11 terminal and introduces the real LangChain screenshot. Hai demonstrates an out-of-PDF question, enters the instructor's questions, and closes the presentation. Both can assist with the live Q&A.

Demonstration safeguard

Use only observed outputs and verify answers against the sample PDF. A retrieved source list alone does not establish that the generated answer is supported. The old slides 31–32 become slides 38–39 in the speaking-order edition.

Splitting Text into Chunks

WHAT TO SAY

The text_chunker module uses RecursiveCharacterTextSplitter with a maximum chunk size of 900 characters and an overlap of 150 characters. The code validates that the chunk size is positive and that overlap is smaller than chunk size. It also assigns a chunk ID to each piece. Splitting makes long documents manageable, and overlap helps preserve some context across neighboring chunks. These settings are measured in characters by this implementation, not tokens.

KEY TAKEAWAY

Chunk size and overlap influence retrieval quality.

Chunk Overlap Explained

WHAT TO SAY

Imagine that an important sentence crosses the boundary between two chunks. If we split text without overlap, the retriever may find only half of the idea. Our project uses an overlap of 150 characters, allowing neighboring chunks to share some information. The diagram is conceptual; the exact chunk lengths depend on the text separators and content. Overlap is a useful starting point, but a larger overlap is not always better because it can increase redundancy.

KEY TAKEAWAY

Overlap preserves context at boundaries.

Multilingual Text Embeddings

WHAT TO SAY

An embedding model converts text into a vector of numeric values. Similar meanings can produce vectors that are close together in the embedding space. Our source uses HuggingFaceEmbeddings with paraphrase-multilingual-MiniLM-L12-v2. This supports semantic retrieval when users ask questions in Vietnamese. The numbers on this slide are an illustration of a vector, not values computed from our sample document. The key principle is that document chunks and questions must use a compatible embedding model.

KEY TAKEAWAY

Embeddings let FAISS compare meaning rather than literal words alone.

FAISS Vector Store

WHAT TO SAY

The vector_store module builds a FAISS index from document chunks and their embeddings. It saves the index locally and can load it again for later questions. Saving avoids recomputing embeddings every time. The implementation loads index.pkl with dangerous deserialization enabled, so that saved pickle must come from a trusted source. We should never import an arbitrary index.pkl downloaded from an untrusted website.

KEY TAKEAWAY

FAISS provides local retrieval, and its saved files require care.

Index Caching and Automatic Rebuild

WHAT TO SAY

The pipeline uses a manifest file to decide whether to reuse the saved FAISS index. It records PDF names, sizes, and modification times, as well as the chunk and embedding settings. When the application starts again, it compares the current values with the manifest. If they match and the FAISS files exist, it loads the saved index. Otherwise, it rebuilds the index. This is a convenient classroom feature, although it is metadata-based change detection rather than a full file-content hash.

KEY TAKEAWAY

The application checks whether the index is still compatible.

The Offline Indexing Pipeline

WHAT TO SAY

The indexing pipeline begins by finding PDF files, reading pages with PyPDFLoader, splitting them into chunks, producing multilingual embeddings, and building a FAISS index. The pipeline module coordinates these operations and saves the resulting index and manifest. This stage is performed when the documents or relevant settings change. We can explain the pipeline directly from four source modules instead of describing an abstract architecture unrelated to our code.

KEY TAKEAWAY

The index stores searchable representations of PDF text.

The Question-Answering Pipeline

WHAT TO SAY

When the user types a question, the system retrieves up to four similar chunks from the saved FAISS index. It formats those chunks with source information and inserts them into a prompt. LangChain then calls the local Qwen2.5 model through ChatOllama and converts the model output to a string. The application prints both the answer and the retrieved document metadata. These are related but separate outputs; a displayed source list is not automatically proof that the answer is correct.

KEY TAKEAWAY

Question → retrieval → context → model → answer and sources.

Retrieving Documents

WHAT TO SAY

The retrieve_documents function calls the FAISS retriever with search_kwargs containing k. The active value is four, so it requests up to four documents. If the user enters an empty question, the function returns an empty list. Retrievers rank similarity, but the top result may not contain sufficient evidence for a specific claim. Therefore, retrieval relevance and answer faithfulness must be evaluated separately.

KEY TAKEAWAY

Top-k controls how many chunks are requested, not their correctness.

Context Formatting

WHAT TO SAY

The format_context function labels each retrieved chunk with a number, file name, and page number before joining it into a context string. The prompt then places the question beside the context. This structure can help the model refer to source passages using bracketed numbers. However, the actual terminal output shows that the model does not always include these inline citations, even though the prompt requests them.

KEY TAKEAWAY

Evidence labels help, but citations still need verification.

LangChain Expression Language

WHAT TO SAY

The new implementation genuinely uses LangChain Expression Language. In generate_answer, the program creates a ChatOllama instance with temperature zero and combines a prompt template, the model, and StrOutputParser into a chain using the pipe operator. It then invokes that chain with the question and formatted context. This is a major technical difference from our old deck, which described a direct ollama.chat call. The slide now matches the actual source.

KEY TAKEAWAY

The code uses PROMPT | ChatOllama | StrOutputParser.

Prompt Instructions

WHAT TO SAY

The system prompt in src/qa.py tells the model to answer in Vietnamese, rely on the provided PDF context, and say when the answer is unavailable. It requests inline references such as [1] and [2]. It also instructs the model not to follow malicious instructions embedded inside PDF text. These are useful safeguards. However, prompt instructions are not guarantees, as our real screenshot illustrates: a plausible definition of LangChain appears even though the bundled PDF does not supply the definition.

KEY TAKEAWAY

Prompt grounding rules are helpful but imperfect.

SPEAKER HANDOFF / LIVE DEMO LINE

Now Nguyen will discuss the local runtime, evaluation, and practical limitations.